**LAB 3 (SECJ1023)**

**PROGRAMMING TECHNIQUE II**

**SECTION 03, 04 & 06, SEM 2, 2025/2026**

1. An ellipse can be represented by its center point (*x, y*), horizontal radius, *rx* and vertical radius, *ry* as shown in Figure 3.1. A circle is a special type of an ellipse in which *rx* and *ry* are the same length. Given the definition of a class representing an ellipse in Program 3.1. Answer questions (a) to (c).

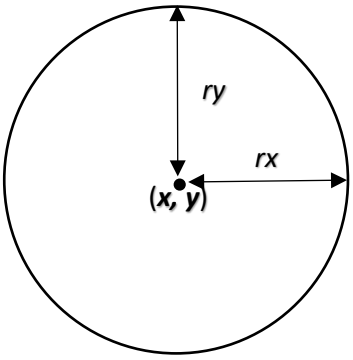



**Figure 3.1:** Ellipse

```
//Program 3.1
class Ellipse{
   private:
      int x, y;
      int rx, ry;

   public:
      Ellipse() {x=y=rx=ry=0;}
      Ellipse(int r) { x=y=0; rx=ry=r;}
      Ellipse(int _rx, int _ry) {x=y=0; rx=_rx; ry=_ry;}

      Ellipse(int _x, int _y, int r)
      {x=_x; y=_y; rx=ry=r;}

      Ellipse(int _x, int _y, int _rx, int _ry)
      { x=_x; y=_y; rx=_rx; ry=_ry;}

      void setRadius(int _rx, int _ry){rx=_rx; ry=_ry;}
};
```

a. In general, what is the purpose of having several constructors in a class?

b. Write five different code in which each code will be creating a circle with the center at the **`origin(0,0)`** and the radius of 10 unit from the class **`Ellipse`**.

c. What if the class **`Ellipse`** is added with another constructor as given below? Justify your answer.

```
Ellipse(int _x, int _y)
{x=_x; y=_y; rx=ry=0;}
```

2. For a quadratic equation $a\,x^2 + b\,x + c = 0$ (where a, b and c are coefficients), its roots is given by following the formula:

$$x1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

and

$$x2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

The program will ask the user to enter coefficients a, b and c and computes the roots of a quadratic equation. The computation should follow the following **`class`** specification:

**`private`** members:

| | |
|---|---|
| **`a, b, c`** | type **`double`** coefficients values |
| **`discriminant`** | type **`double`** determinant value |
| **`calcDiscriminant()`** | function with return inline function member to calculate the determinant value, $b^2 - 4ac$ |

**`public`** members:

| | |
|---|---|
| **`Compute(double, double, double)`** | a constructor that takes arguments 3 arguments to initialise the coefficients value, and invoke **`calcDiscriminant()`** to **`obtain`** the determinant value |
| **`positive_root()`** | function to calculate and return $x1$ value |
| **`negative_root()`** | function to calculate and return $x2$ value |

a. In general, what is the purpose of having several constructors in a class?

b. Define a class **`Compute`** based on the specifications given above.

c. Write the definition of the constructor **`Compute (double, double, double)`** to initialize the private member variable **`a, b, c`** to the value of the parameter, and assign the private member variable **`discriminant`** to invoke **`calcDiscriminant()`** function.

d. Based on the roots of quadratic equation given above, define two function members for **`positive_root()`** and **`negative_root()`** to calculate $x1$ and $x2$ values.

e. The following **`main()`** function of Program 3.2 is incomplete. Fill in the three missing statements according to the instructions as commented in (a) to (c).

```
//Program 3.2
int main()
{
   double a, b, c;
   cout << "Enter coefficients a, b and c: ";
   cin >> a >> b >> c;
   // (a)Pass arguments a, b, c to the constructor
   ___________________________;
```

```
    cout << "The roots of the quadratic equation are x = ";
    // (b)Execute function members to display the value x1
    cout << ________________________;
    cout << ", ";
    // (c)Execute function members to display the value x2
   cout << _________________________ << endl;

   return 0;
}
```

f. The **`main()`** function is completed and you have compiled them without errors.

   i. What is the output of the program if the inputs entered by the user is **`a = 4`**, **`b = 5`** and **`c = 1`**.

   ii. Based on the result you found in (e), what is the value of **`discriminant`**? Assume the condition given in Pseudocode 1 is working in the program, what is the output that will be printed by the program?

   **Pseudocode 1:**

```
If (discriminant = 0)
      Print "The equation has a single root."
Else if (discriminant < 0)
      Print "The equation has two complex roots."
Else
      Print "The equation has two real roots."
```

3. There are **10 lines** with syntax errors in the Program 3.3. Identify each of the errors and suggest the cause, and finally write the correct code solve the syntax errors. Describe the function of the main function in the corrected code version:

```
//Program 3.3
#include<iostream>
#include<string>
using namespace std;

class Student
{
    private:
       string name;
    public:
       void Student() { name = ""; }
       Student(string aName = "") { name = aName; }
       ~Student(string msg)
       { cout << "The object has been destroyed" << endl; }

       string getName() const; { return name; }
       void setName(string aName) const { name = aName; }
};

int main()
```

```
{
    student Student("Siti Aminah");
    Student s1="Ramlee Puteh";
    Student s2("Ahmad Mahmood");
    Student s3;

    cout << "Student's name: " << s2.name << endl;
    s3.setName = "Mustafa Kamal";
    s2.name = s1.name;

    return 0;
}
```

4. Consider the **`FeetInches`** class definition in Program 3.4.

```
//Program 3.4
class FeetInches
{
   private:
      int feet;
      int inches;

   public:
      FeetInches(int f=0, int i=0) {
         feet = f;
         inches = i;
      }

      void setFeet(int f) { feet = f; }
      void setInches(int i) { inches = i; }
      int getFeet() const { return feet; }
      int getInches() const { return inches; }
};
```

a. Add a copy constructor to the **`FeetInches`** class. This constructor should accept a **`FeetInches`** object as an argument. The constructor should assign to the **`feet`** attribute the value in the argument's **`feet`** attribute, and assign to the **`inches`** attribute the value in the argument's **`inches`** attribute. As a result, the new object will be a copy of the argument object.

b. Add a **`multiply`** member function to the **`FeetInches`** class. The **`multiply`** function should accept a **`FeetInches`** object as an argument. The argument object's **`feet`** and **`inches`** attributes will be multiplied by the calling object's **`feet`** and **`inches`** attributes, and a **`FeetInches`** object containing the result will be returned.

c. Implement your **`FeetInches`** class with the following **`main`** function:

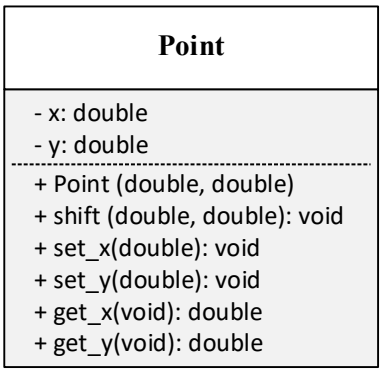
```
int main()
{
   FeetInches one(20,50);
   FeetInches two(one);
   FeetInches three;
```

```
    three = one.multiply(two);

    cout << one.getFeet() << endl << two.getFeet() << endl
         << three.getFeet() << endl;

    return 0;
}
```

5. Based on the following class diagram, write a complete C++ program that do the following tasks.




a. Write the constructor for the class **point** that initialize the values of **`x`** and **`y`** with the passed arguments. This constructor also enables default constructor to initialize both values **`x`** and **`y`** to 0.

b. Define the mutator and accessor functions.

c. Write definition of the member function **`shift()`** that modify values of **`x`** and **`y`** by summing up with the passed arguments, **`x_amount`** and **`y_amount`**, respectively.

d. Write the **`main`** function to create two instance object **`p1`** with parameter setting and **`p2`** with default constructor. Set **`p2`** coordinate with mutator functions and call **`shift()`** function to move the two coordinates to move other quadrant. Make sure the coordinates are printed before and after the move operation.

6. Write a complete C++ program that contains a class called **`Product`**. This class is used for holding data about products sold by a grocery store. The class should have the following **`private`** member variables (attributes):

| Attribute | Description |
|---|---|
| `Code` | is of type `int` that holds the product's code number. |
| `Price` | is of type `double` that holds the price of the product. |

The class should also have the following **`public`** member variables (methods):

| Attribute | Description |
|---|---|

| | |
|---|---|
| Default constructor | Sets all the attributes to **0**. |
| Overloaded Constructor | Sets all the attributes from arguments. |
| Destructor | Does nothing. |
| **setCode** | is a mutator function that sets the **code** attribute from an **int** argument. |
| **setPrice** | is a mutator function that sets the **price** attribute from a **double** argument. |
| **getCode** | is an accessor function that returns the value of **code** attribute. |
| **getPrice** | is an accessor function that returns the value of **price** attribute. |

In the **main** function of the program:

a. Declare an array to hold a list of up to 100 objects of class **Product**.

b. Ask the user to enter the number of products to key in.

c. Set the attributes of each object in the array using the mutator functions. Data to assign are firstly read from the keyboard. Your program is expected to use a loop for doing this.

d. Add another Product object to the array. The objects's code and price attributes are set to 100 and 1.00 respectively. Your program is expected to use the overloaded constructor for creating this object.

e. Finally, print the product information in a table as shown in the Figure 3.2. Your program should use another loop for doing this part. Your program should also calculate the average price of the products and print it at the end of the table.

The **bold text** in the following figure represent user inputs from the keyboard.

```
How many products to enter? 2

Enter the information of product #1
Product Code: 101
Price: 4.50

Enter the information of product #2
Product Code: 933
Price: 8.90

No      Product Code    Price

1       101             4.50
2       933             8.90

Average Price:  RM 6.70
```

**Figure 3.2**

7. Define a **`SavingsAccount`** class which has the following:

   a. static member variable, **`annualInterestRate`**

   b. **`savingBalance`** indicating the amount the saver currently has on deposit.

   c. **`calculateMonthlyInterest`**function that calculates the monthly interest by multiplying the balance by **`annualInterestRate`** divided by 12. This interest should be added to **`savingBalance`**.

   d. Static member function **`modifyInterestRate`** that sets the static **`annualInterestRate`** to a new value.

   Write a main function to test class **`SavingsAccount`**.

   a. Instantiate two different **`SavingsAccount`** objects, **`saver1`** and **`saver2`**, with balances of RM2000.00 and RM3000.00, respectively.

   b. Set **`annualInterestRate`** to 3%, then calculate the monthly interest and print the new balances for each of the savers.

   c. Then set the **`annualInterestRate`** to 4% and calculate the next month's interest and print the new balances for each of the savers.

8. Define a **`Mechanic`** class which has the following:

   a. Member variables: **`name, staffid, hoursworked, salary, rate per hour`**.

   b. Constructor that accepts the mechanic **`name, staff id, hours worked`** and **`rate per hour`**. These values should be assigned to the object's name, staff id, hours worked and rate per hour member variables.

   c. **`calculateSalary`**: This function will calculate salary of the mechanic based on the formula: **`salary = total hours worked * rate per hour.`**

   d. **`display`**: This function will display all mechanic's information along with his salary.

   e. **`calculateEPF`**: This is a friend function. This function will receive a Mechanic object as its argument. This function will calculate and return the **`epf`**for the object. The calculation for the **`epf`**: **`epf = 0.05 * salary.`**

   Demonstrate the class in a **`main`** function that:

   a. Creates a **`Mechanic`** object then,

   b. Calculate the mechanic salary and display his information.

   c. Display the mechanic's **`epf`** contribution value.